

WHITEPAPER

OnChain Poker Whitepaper

A Provably-Fair, Dealer-Free Protocol for 2-6 Handed NLHE and PLO5 on Polygon

OnChain Poker is a fully decentralized, provably-fair poker protocol that runs entirely inside a smart contract on Polygon. Two to six players shuffle, encrypt, and deal their own cards using a generalized mental-poker cryptographic protocol — with no dealer, no casino operator, and no external randomness oracle of any kind. Deposits, buy-ins, betting, side pots, and payouts are all self-custodial on-chain transactions that settle the instant they happen, with no cashier and no withdrawal queue. The protocol supports both No-Limit Hold'em and 5-card Pot-Limit Omaha, at heads-up or six-max tables, and every step of every deal can be independently verified after the fact by anyone. This is a beta release — see Section 7 for what that means in practice.

Beta Release Whitepaper — v1.0

Polygon (POL) · Smart-contract custody · Self-custodial settlement

Table of Contents

Abstract

1. Introduction

- 1.1 The problem with “trust us” poker
- 1.2 What this protocol supports
- 1.3 No randomness oracle — and why that’s a deliberate choice, not a gap
- 1.4 Scope of this document

2. System Overview

- 2.1 Actors
- 2.2 High-level lifecycle of a hand
- 2.3 Custody and the chip ledger
- 2.4 Self-custodial, with no cashier and no wait

3. Table and Seating Model

- 3.1 Modes
- 3.2 Game type
- 3.3 Lobby transparency

4. Cryptographic Foundations: Mental Poker Without a Dealer

- 4.1 The building block: commutative encryption
- 4.2 The protocol, step by step, for N players
- 4.3 Where the fairness actually comes from
- 4.4 Verifiability: the other half of “provably fair”
- 4.5 Summary table: the four fairness properties

5. On-Chain State Machine

6. The Betting Engine

- 6.1 Seat-order action
- 6.2 Heads-up vs. 3+ handed blind/action order
- 6.3 Side pots
- 6.4 Pot-limit betting (PLO5 only)
- 6.5 Uncalled bets
- 6.6 Timeouts and liveness

7. Known Limitations and Trust Assumptions

- 7.1 This is a beta release
- 7.2 Folded players must keep cooperating on community cards
- 7.3 Betting-engine simplifications
- 7.4 The choice of prime modulus
- 7.5 Client-side randomness and passphrase hygiene
- 7.6 No protection against a single dishonest majority conspiring *before* the table is seated
- 7.7 Regulatory status

8. Economic Design

- 8.1 Chip unit and stake ladder
- 8.2 Rake

9. Function Reference (Summary)

10. Conclusion

Appendix A: Slot Numbering Formulas

Appendix B: Glossary

Abstract

OnChain Poker is a smart-contract-based poker protocol that removes the dealer, the house, and the RNG oracle from the game entirely. Card shuffling and dealing are performed by the players themselves, using a generalization of the classical **SRA mental-poker protocol** (Shamir, Rivest, Adleman, 1979) to tables of two to six seats. Every step of the deal — the shuffle, the encryption, every card reveal, and the final showdown — is a transaction or a value recorded on the Polygon blockchain, and every one of those values can be independently re-derived and checked by anyone after the fact. No party ever has to be trusted to deal honestly; if anyone tries to cheat, the contract can prove it happened and cheaters forfeit the pot.

Because dealing needs no external input, the protocol also needs **no randomness oracle of any kind** — no third-party random-number service, on-chain or off. This is a deliberate design choice, not an omission: an oracle would reintroduce exactly the kind of centralized, hard-to-audit trust point (an external operator or node network whose honesty and timing a player cannot personally verify, and whose fairness has no mathematical proof — only a claim) that mental poker’s own cryptography is built to make unnecessary. The same self-contained design gives players **self-custodial, on-chain deposits and withdrawals** — no cashier, no manual approval queue, no multi-day withdrawal review — and **payouts that settle the instant a hand resolves**, not on an operator’s schedule.

This document describes the complete system: the cryptographic protocol, the on-chain state machine, the betting engine (including side pots and pot-limit betting for Omaha), the two supported game types (No-Limit Hold’em and 5-card Pot-Limit Omaha), the seating and table model (heads-up or six-max), and — in detail — exactly what “provably fair” means here and what guarantees it does and does not give you. This is a **beta release**, functionally complete and consistent with this document — see §7.1 for what that means in practice.

1. Introduction

1.1 The problem with “trust us” poker

Every online poker room that has ever operated has asked its players to trust a private, off-chain random number generator to shuffle the deck honestly, and to trust the operator not to peek at hole cards, not to seed the RNG with a bias, and not to quietly settle disputes in its own favor. Reputable operators publish RNG certifications from third-party labs, but the player is still trusting a certificate, not verifying a specific hand.

Blockchain-based poker removes the operator from the trust boundary in two ways:

1. **Custody** — chips are held by a smart contract, not a company, and payouts are enforced by code that anyone can read.
2. **Dealing** — with no dealer at all, the players themselves jointly shuffle and encrypt the deck such that no single party (including any subset short of everyone) can know the full card arrangement in advance, and every step of the process is either an on-chain transaction or a value that can be checked against on-chain commitments after the hand.

This second property — dealing without a dealer — is the harder problem, and it is the subject of most of this document. Mental poker itself dates to a 1979 paper by Shamir, Rivest, and Adleman; what this protocol contributes is carrying that construction, in full, onto a live blockchain — the shuffle, every reveal, and the showdown verification are all on-chain transactions, not an off-chain protocol that merely settles on-chain — and generalizing it from its original two-player form to tables of up to six.

1.2 What this protocol supports

- **Table sizes:** heads-up (2 seats) or six-max (up to 6 seats), chosen once by whoever creates the table.
- **Game types:** No-Limit Hold’em (NLHE) or 5-card Pot-Limit Omaha (PLO5), also

chosen once at table creation.

- **Custody:** a chip ledger inside the contract, funded and cashed out via native POL on Polygon at a fixed exchange rate.
- **Settlement:** automatic, trustless payout at the end of every hand, including correct side-pot math for uneven all-ins and a fixed 1% rake.

1.3 No randomness oracle — and why that’s a deliberate choice, not a gap

Most on-chain games that need an unpredictable outcome — dice, roulette-style contracts, loot boxes — still depend on an external randomness oracle service, because a smart contract cannot generate genuine randomness by itself; it can only compute deterministically on the inputs it’s given. This is a common pattern, but it does not eliminate trust from the system — it **relocates** it, from a casino operator to whichever third-party service is providing the number: its operators, its funding and subscription mechanics, and the timing of exactly when a request is fulfilled relative to other on-chain activity. A player is still being asked to trust a party they cannot see inside, just a different one, and there is no mathematical proof that party is behaving fairly — only a claim, backed by reputation rather than verifiable cryptography. That party also sits outside the game’s own rules and could in principle be delayed, priced out of service, or otherwise unfairly positioned relative to the players themselves — the exact opposite of decentralization.

Poker does not have this problem, because dealing a deck is not “produce one random number” — it is “arrive at a shuffle that no single participant controls or can predict alone,” and that is exactly what the mental-poker construction in Section 4 achieves using nothing but the players’ own secrets. There is no request to an outside service anywhere in the deal, the reveal, or the showdown. **OnChain Poker uses no oracle, and no third-party randomness service of any kind, for any part of the game, by design** — not because integrating one would be difficult, but because doing so would reintroduce a centralized point of trust and potential manipulation that the cryptography in Section 4 was specifically built to remove. The players are the entire source of unpredictability, no outside party is ever trusted with anything, and Section 4.3 states precisely how strong a guarantee that gives you.

1.4 Scope of this document

This whitepaper describes the reference implementation exactly as written (`OnChainPoker.sol` and its companion web client). It is deliberately conservative in its claims: Section 7 lists, without euphemism, the properties this design does **not** give you, because a whitepaper that only describes the happy path is not a useful whitepaper.

2. System Overview

2.1 Actors

- **Players** — any address holding chips who sits at a table.
- **The contract** — `OnChainPoker.sol`, the sole custodian of chips and the sole arbiter of game state. It has no owner-only game logic; the only privileged action in the entire contract is rotating the rake-collection address, and only the current rake recipient can do that.
- **Nobody deals.** There is no dealer role, no oracle, and no off-chain server in the trust path. The reference web client is a convenience for doing the client-side arithmetic; a player could in principle interact with the contract directly.

2.2 High-level lifecycle of a hand

```
create table → players sit → start hand → post blinds
  → N-way deck encryption chain → hole-card layer reveals
  → betting rounds (repeat per street, with community-card reveals
    between streets) → showdown key reveal & verification
  → side-pot payout
```

Every arrow above is either an on-chain transaction from a player, or an automatic state transition inside the contract triggered by the last required transaction (e.g. the contract moves itself from “awaiting deck” to “awaiting hole reveals” automatically the moment the last player in the encryption chain submits).

2.3 Custody and the chip ledger

Chips are a purely internal unit of account: `1 chip = 0.001 POL`, fixed in the contract as `CHIP_UNIT_WEI`. `deposit()` converts POL to chips at that rate (refunding any dust); `withdraw()` converts back. A table’s buy-in must be at least 50× its big blind, and the big blind itself must be one of seven fixed tiers (2, 5, 10, 50, 200, 500, 1000 chips), so every table in the lobby represents a known, comparable stake — there is no way to create an off-ladder game.

2.4 Self-custodial, with no cashier and no wait

Because the contract itself is the custodian, there is no operator-run cashier, no manual withdrawal review, no minimum hold period, and no queue behind other players’ pending requests. `deposit()` and `withdraw()` are single transactions a player submits themselves, at any time, to their own wallet — nobody else’s approval is required and nobody else can delay, cap, or reverse them. Funds move as soon as that transaction confirms on Polygon, typically a matter of seconds, not on a business’s payout schedule. Hand winnings are likewise credited to a player’s table stack automatically the instant a hand resolves — there is nothing to request and nothing to wait on beyond ordinary blockchain confirmation.

3. Table and Seating Model

3.1 Modes

Mode	Seats	Fill requirement to start a hand
<code>HEADS_UP</code>	exactly 2	both seats occupied
<code>SIX_MAX</code>	up to 6	at least 2 of the 6 occupied — the table never has to be full

The creator of a table chooses the mode permanently at `createTable(...)`. Six-max tables can start, run, and finish hands while seats are still open; other players can sit down at any empty seat (`joinTable(tableId, seatIndex)`) between hands, including while a hand is already in progress at that table — the new player simply isn’t dealt in until the *next* hand starts.

3.2 Game type

Game type	Hole cards	Hand selection rule	Betting cap
<code>NLHE</code>	2 per player	best 5 of any 7 (2 hole + 5 board)	no limit (up to your stack)
<code>PL05</code>	5 per player	exactly 2 hole + exactly 3 board	pot-limit (see §6.4)

This is also chosen once, permanently, at table creation.

3.3 Lobby transparency

`getTableSummary(tableId)` returns, in one call: the mode, the game type, the number of seats filled versus the table’s maximum, every seat’s occupant and stack, the stakes, and whether a hand is currently in progress. A lobby UI can therefore show, e.g., “*Table #7 — Six-max · PLO5 · 3/6 players · hand in progress*” and still let a spectator sit down at one of the three open seats without waiting for the current hand to finish.

4. Cryptographic Foundations: Mental Poker Without a Dealer

4.1 The building block: commutative encryption

The entire scheme rests on one algebraic fact: **modular exponentiation to a shared prime modulus commutes**, regardless of who chose which exponent.

For a fixed prime P and any message $m \in \mathbb{Z}_P^*$, if two parties pick exponents e_1 and e_2 independently, then:

$$(m^{e_1} \bmod P)^{e_2} \bmod P = (m^{e_2} \bmod P)^{e_1} \bmod P = m^{e_1 e_2} \bmod P$$

This means that if N players each encrypt a value with their own secret exponent, one after another, the **order in which those layers are later removed does not matter** — any subset of the layers can be stripped off in any sequence, and the result is exactly as if the remaining layers had been the only ones applied. This is the 1979 Shamir-Rivest-Adleman (“SRA”, also called the “three-pass protocol” in its two-party form) mental-poker construction, generalized here from two players to up to six.

Each player additionally chooses an RSA-style key pair (e_i, d_i) with $e_i \cdot d_i \equiv 1 \pmod{P-1}$, so that raising to e_i and then to d_i returns the original value — this is how a layer is *removed* rather than merely re-scrambled.

The prime used is:

```
P = 57896044618658097711785492504343953926634992332820282019728792003956564819949
```

— the well-known Curve25519 field prime ($2^{255} - 19$), chosen for a well-audited, widely-verified value and reasonable gas cost via the EVM’s `modexp` precompile (address `0x05`). Its use here is purely as a large prime modulus for classical discrete-log-based encryption; see §7.4 for a specific caveat about this choice.

4.2 The protocol, step by step, for N players

Let the players seated for a given hand be numbered $0, 1, \dots, N-1$ in ascending table-seat order. Let `holeCardCount` be 2 for NLHE or 5 for PLO5.

Step 1 — Canonical deck. The deck is simply the integers 1..52. Rank and suit are pure functions of the integer (`rank = ((card-1) % 13) + 2`, `suit = (card-1) / 13`); there is nothing to shuffle yet, because nobody has touched it.

Step 2 — Sequential shuffle + encryption. Player 0 privately picks a secret random permutation π_0 of the 52 positions and a secret key pair (e_0, d_0) . For every position i :

$$\text{deck}_1[i] = (\pi_0(i)+1)^{e_0} \bmod P$$

and submits the resulting 52-element array on-chain via `submitEncryptedDeck`. **This array is player 0’s irrevocable commitment** — nothing about it can be changed after submission, and nobody, including player 0, can produce a different permutation later that is still consistent with this array (see §4.4, verifiability).

Player 1 then takes that array, applies their **own** secret permutation π_1 and key pair (e_1, d_1) on top:

$$\text{deck}_2[i] = \text{deck}_1[\pi_1(i)]^{e_1} \bmod P$$

and submits that. This repeats through player $N-1$. The final array, deck_N , is encrypted under *all* N players’ keys and shuffled by the composition of all N permutations. **No player ever sees a plaintext value belonging to another player’s shuffle step** — every submission is ciphertext from start to finish.

Step 3 — Fixed slot layout. For a hand with N players, hole-card slots occupy positions $0 \dots (\text{holeCardCount} \times N - 1)$, laid out player-by-player (player i ’s hole cards are slots $\text{holeCardCount} \cdot i, \dots, \text{holeCardCount} \cdot i + \text{holeCardCount} - 1$). Immediately after

come the flop (3 slots), the turn (1 slot), and the river (1 slot). Any remaining positions up to 51 are unused “burn” slots that are never revealed and therefore carry no information either way.

Step 4 — Revealing a player’s own hole cards. To let player i see their own cards, **every other player in the hand** removes their own encryption layer from those slots, in any order (commutativity guarantees this is safe), by applying their own d_k and submitting the intermediate result (`revealHoleLayer`). Once all $N - 1$ other layers are off, the value on-chain is encrypted under player i ’s key alone — and only player i , holding d_i privately, can finish the decryption, client-side, off-chain. **No other player, including all $N - 1$ of the others acting together, can determine player i ’s cards from this exchange**, because doing so would require d_i , which player i never reveals until the hand is completely over (see §7.2 for the one important caveat to this statement).

Step 5 — Revealing community cards. Flop, turn, and river slots require **all N players** — including anyone who has since folded — to remove their layer before the card is plaintext. By convention the *last* player to remove a layer for a slot submits the fully-decrypted value (1–52) directly, rather than a further-encrypted intermediate value; the contract simply records it once all N removals are accounted for.

Step 6 — Folding. If a hand narrows down to one remaining (non-folded) player at any point, that player wins the entire pot immediately, with any portion of their own bet that nobody could possibly have called returned to them untaxed by rake. No keys or cards need ever be revealed for a fold-out.

Step 7 — Showdown. If two or more players reach the river without folding, **every one of the N original players in the hand — folded or not — reveals their full key pair and permutation** (`revealShowdownKeys`). The contract then replays the entire N -stage encryption chain using the EVM’s `modexp` precompile and checks that every stage’s on-chain array is exactly what the revealed keys and permutation would produce. This is the verification step, covered in detail in §4.4.

Step 8 — Fresh keys every hand. All keys and permutations are derived fresh per hand from a monotonically increasing per-table hand counter (`handNonce`), so revealing them at showdown never weakens the secrecy of any other hand, past or future.

4.3 Where the fairness actually comes from

It is worth stating precisely, because “provably fair” is a phrase that gets used loosely elsewhere: the guarantee is **not** “the outcome is drawn from a certified random source.” It is the following, stronger and more specific, statement:

As long as at least one of the N players keeps their permutation and key secret and applies it honestly, **no coalition of the remaining $N - 1$ players — even if they fully collude and pool every secret they hold — can determine the plaintext value at any card position**, because every position’s final value is a function of every player’s secret exponent and permutation applied in sequence. A colluding majority is missing exactly the one input it does not have, and modular exponentiation gives no shortcut around a missing exponent (this is the same discrete-logarithm hardness assumption RSA and Diffie–Hellman rely on).

This is a much stronger property than “we used a certified RNG”: it holds even if every player *except one* is actively malicious and coordinating, as long as that one honest player follows the protocol. It degrades gracefully to nothing only in the degenerate case where literally every player at the table colludes against a party outside the hand — which is meaningless, since there is no such party; the players *are* the ones with money on the table.

4.4 Verifiability: the other half of “provably fair”

Secrecy during the hand is only half the property. The other half is that **after the hand, anyone can check that nobody cheated during it**. This is what the showdown reveal is for.

When a player reveals (e_k, d_k, π_k) at showdown, the contract:

1. Confirms $e_k \cdot d_k \equiv 1 \pmod{P-1}$ (a valid key pair) and that π_k is a genuine permutation of 0..51 (no repeats, no out-of-range values).
2. Replays player k 's claimed transformation from the previous stage's on-chain array (or the canonical deck, for player 0) and checks, position by position, that it reproduces **exactly** the array player k submitted during the deal. This uses the same `modexp` precompile the deal itself used, so verification is deterministic and exact — not a probabilistic check.
3. If a player's reveal does **not** reproduce their earlier submission, that is on-chain proof that the number they submitted during the deal was not, in fact, the honest output of the process they now claim — proof of tampering. That player automatically forfeits: they are treated as folded for the rest of the payout calculation, and a `CheatDetected` event is emitted naming them.

Because every submission during the deal was already an irrevocable, timestamped, on-chain commitment, a cheater's only options are (a) play honestly, or (b) get caught with mathematical certainty the moment they try to prove otherwise. There is no way to submit a dishonest array during the deal and later "explain it away" with a different key — the check in step 2 above is exact equality over all 52 positions, not a spot check.

4.5 Summary table: the four fairness properties

Property	Mechanism	Failure mode if violated
Hiding	Commutative SRA encryption; a card is plaintext only once every relevant player's layer is stripped	Would let a party see cards before authorized to
Binding	Every deal-stage array is submitted as an immutable on-chain transaction before any card is used	Would let a party retroactively change the shuffle
Verifiability	Showdown key reveal + exact <code>modexp</code> replay of the whole chain	Would let a cheater submit a bad array and never get caught
Liveness	Timeouts at every stage (see §6.6) force resolution even if a party goes silent	Would let a party stall a hand indefinitely to avoid losing

5. On-Chain State Machine

Each table has exactly one active hand at a time, represented by a `Stage` :

Stage	Meaning	Who can act
<code>WAITING_FOR_PLAYERS</code>	No hand running	Anyone seated may call <code>startHand</code> once ≥ 2 seats are filled
<code>AWAITING_DECK</code>	The N-way encryption chain is in progress	Whoever is next in the chain (<code>players[deckSubmissions]</code>)
<code>AWAITING_REVEALS</code>	All N deck layers submitted; hole-card layer removal in progress	Any player, removing others' layers
<code>BETTING</code>	Standard betting round for the current street	Whoever's turn it is (<code>toActIdx</code>)
<code>AWAITING_SHOWDOWN_KEYS</code>	River complete, 2+ players still in; keys being revealed	All N original players (folded or not)
<code>RESOLVED</code>	Hand is over; pot distributed	— (call <code>startHand</code> again)

Streets (`PREFLOP` → `FLOP` → `TURN` → `RIVER` → `SHOWDOWN`) advance automatically the instant every player who can still act has matched the current bet, or automatically cascade straight through to showdown if every remaining player is already all-in (a "runout").

6. The Betting Engine

6.1 Seat-order action

Betting proceeds in table-seat order among the players dealt into the current hand, skipping anyone folded or already all-in. Blinds and the button rotate to the next *occupied* seat every hand — a six-max table with empty seats simply skips over them.

6.2 Heads-up vs. 3+ handed blind/action order

Players	Small blind	Big blind	First to act preflop	First to act postflop
2 (heads-up)	Button	Non-button	Button (small blind)	Non-button
3+	Seat after button	Seat after SB	Seat after BB (UTG)	Seat after button

6.3 Side pots

Because players can be all-in for different amounts, the contract computes side pots using the standard layered algorithm: it collects every distinct total-committed level among all original players in the hand, and for each layer, awards that slice of the pot to the best-scoring hand among only the players who committed at least that much and never folded. This is computed fully on-chain at showdown — there is no off-chain pot-splitting logic to trust.

6.4 Pot-limit betting (PLO5 only)

For PLO5 tables, every bet or raise is capped at the pot-limit maximum:

$\text{max total bet} = \text{currentBet} + (\text{pot} + \text{amount needed to call})$

This is the standard Omaha pot-limit formula: the most you may raise to is the size the pot would be immediately after you called it. NLHE tables have no such cap — a player may bet any amount up to their remaining stack.

6.5 Uncalled bets

If folds bring a hand down to one remaining player, any part of that player's bet that nobody else could possibly have called (because everyone else folded, or was already all-in for less) is returned to them directly — untaxed by rake, exactly as in live poker, rather than being swept into the "pot."

6.6 Timeouts and liveness

Every stage a hand can be in has an associated deadline (`ACTION_TIMEOUT` = 5 minutes, `REVEAL_TIMEOUT` = 10 minutes, `DEAL_TIMEOUT` = 3 minutes) and a defined resolution if it passes:

- **Silent during the deal or hole-card reveals:** the stalling player forfeits only what's already in the pot (the blinds), split evenly among the other players in the hand.
- **Silent during betting:** the stalling player is treated as folded.
- **Silent at showdown:** any player who hasn't revealed their keys by the deadline is treated as folded for payout purposes. If that player had *already* folded earlier and simply never reveals (see §7.2), the remaining pot is split evenly among the players still contesting it rather than risk paying out based on an unverifiable chain.

No player can ever voluntarily leave a table while a hand is active — the only ways out of an active hand are to play it to completion, or to have another player claim a timeout once a deadline passes.

7. Known Limitations and Trust Assumptions

A whitepaper that omits this section is not describing a real system. These are documented directly in the contract's source comments as well.

7.1 This is a beta release

`OnChainPoker.sol` is a **beta release**: it is feature-complete and internally consistent with this document, but it has not yet gone through a professional third-party security audit or extended mainnet track record. Generalizing a two-party mental-poker scheme to up to six players materially increases the trusted-computing surface — the N-way encryption chain, multi-way betting order, and side-pot math are all new relative to the original heads-up design, and each is a place a subtle bug could hide. **Treat it accordingly: start on testnet, run it at low stakes before anything else, and don't deposit funds you cannot afford to lose while it's still in beta.**

7.2 Folded players must keep cooperating on community cards

This is the single most important limitation introduced by going beyond two players. In heads-up poker, a fold ends the hand immediately, so a folded player's cooperation is never needed again. In a 6-max hand, however, a player who folds on the flop may still be needed to help reveal the turn and river for the players still contesting the pot — because their encryption layer is physically part of the *shared* deck array, not something tied only to their own cards. A folded player who simply closes their browser can stall the remaining players.

The contract's timeout mechanism provides a way out (§6.6), but it cannot magically produce a value only the absent player could supply; in the worst case, the remaining pot is split evenly rather than resolved by hand strength, and the absent player is flagged via a `CheatDetected` event for anyone auditing the chain. This is an inherent tradeoff of decentralized multi-party mental poker, not a bug — but it is exactly the kind of thing that should be weighed against the convenience of six-max before real money is involved.

7.3 Betting-engine simplifications

A few standard-poker edge cases are simplified relative to a full casino-grade engine:

- An all-in raise that is **less** than a full minimum raise does not, strictly speaking, need to re-open the betting for players who already matched the previous bet — real casino rules deny them the right to re-raise (only to call or fold) in that specific case. This implementation does not enforce that narrower restriction.
- Community-card reveal timing (exactly when between betting rounds it happens) is left to client-side sequencing convention rather than being enforced as a discrete on-chain stage of its own.

Neither of these affects the cryptographic fairness of the deal; they are betting-rule fidelity notes.

7.4 The choice of prime modulus

The modulus P used for the SRA encryption is the Curve25519 elliptic-curve field prime. This is a well-vetted 255-bit prime, but note that its use *here* is in the **multiplicative group** $\mathbb{Z}_P^*$ for classical discrete-log-based encryption — a different mathematical setting than its normal use as an elliptic-curve field prime. The security of this scheme depends on the hardness of the discrete logarithm problem in $\mathbb{Z}_P^*$, which in turn depends on the factorization of $P - 1$ having no unusually small factors that would make a Pohlig-Hellman-style attack practical. This should be explicitly verified (and, if necessary, a purpose-chosen safe prime substituted) as part of any pre-launch audit.

7.5 Client-side randomness and passphrase hygiene

Each player's per-hand permutation and key pair are derived deterministically from a **passphrase** the player enters into the web client, combined with the table ID and hand

number, via SHA-256. This passphrase is explicitly **not** the player’s wallet private key — reusing a wallet key for this purpose would be a serious security mistake, and the client’s UI warns against it. The strength of a given hand’s shuffle secrecy is only as good as the entropy of this passphrase; a weak or reused passphrase is a player’s own risk, not a flaw in the contract, but it is a real operational consideration worth stating plainly.

7.6 No protection against a single dishonest majority conspiring before the table is seated

Section 4.3 shows that no coalition of players can determine a hand’s outcome *during play*, as long as one honest player follows the protocol. It says nothing about players who agree in advance to share their information with each other after the fact (classic multi-accounting or “soft collusion” at the table, e.g. signaling holdings to a confederate) — that is a game-theoretic and operational problem, not a cryptographic one, and no mental-poker scheme can solve it by itself.

7.7 Regulatory status

Operating real-money poker for other people may require a gambling license in the jurisdiction of the operator and the players. That is a legal determination, not a technical one, and this document takes no position on it.

8. Economic Design

8.1 Chip unit and stake ladder

- 1 chip = 0.001 POL (`CHIP_UNIT_WEI = 1015 wei`).
- Big blind must be one of: 2, 5, 10, 50, 200, 500, or 1000 chips (cash-game-style tiers 1-2, 2-5, 5-10, 25-50, 100-200, 500, 1000).
- Buy-in must be at least 50× the chosen big blind.

8.2 Rake

A fixed 1.00% rake (`RAKE_BPS = 100`) is taken from every **contested** pot at the moment it is awarded — whether by fold, showdown, forfeiture, or split. It is never taken from a hand that is voided without a real contest (e.g. an uncalled bet returned, or a deal-stage timeout with no cards ever dealt). The rake accrues to a single `feeRecipient` address’s chip balance, withdrawable through the same `withdraw()` path as any player’s winnings. The rate itself is a hard-coded constant with no setter of any kind; the only privileged action in the whole contract is the current fee recipient rotating that address to a new one (e.g., for key rotation), which cannot change the rate.

9. Function Reference (Summary)

Function	Purpose
<code>deposit()</code> / <code>withdraw(chips)</code>	Convert POL ↔ chips
<code>createTable(bb, buyIn, mode, gameType)</code>	Open a new table; creator sits at seat 0
<code>joinTable(tableId, seatIndex)</code>	Sit at a specific open seat
<code>leaveTable(tableId)</code>	Cash out and stand up (only between hands)
<code>startHand(tableId)</code>	Begin a hand once ≥2 seats are filled
<code>submitEncryptedDeck(tableId, deck)</code>	Submit your layer of the N-way shuffle/encryption chain
<code>revealHoleLayer(tableId, targetIdx, slots, values)</code>	Strip your layer from another player’s hole cards
<code>revealCommunityLayer(tableId, slot, value)</code>	Strip your layer from a community card
<code>checkAction</code> / <code>callAction</code> / <code>betOrRaiseAction</code> / <code>foldAction</code>	Standard betting actions
<code>revealShowdownKeys(tableId, e, d, perm)</code>	Reveal your full key and permutation for

	verification
<code>claimHandTimeout(tableId)</code>	Force-resolve a stage whose deadline has passed
<code>getTableSummary(tableId)</code>	Lobby-friendly snapshot: mode, game type, seats, stacks, stage

(This is a summary; the contract source is the normative reference for exact signatures and behavior.)

10. Conclusion

OnChain Poker replaces the single hardest piece of trust in online poker — “trust that the deal was fair” — with a protocol where fairness is a property you can check rather than a certificate you have to believe. A generalized SRA mental-poker chain lets two to six players shuffle and deal a deck with no dealer, no oracle, and no operator with early sight of any card, while every step remains bound to an on-chain commitment that can be replayed and verified, exactly, after the fact.

That does not make this a finished, production-hardened release — it is a **beta**. Section 7 is not boilerplate — it describes real, specific tradeoffs that come from doing this with more than two parties, and any deployment handling real funds should treat that section as a checklist, not a disclaimer.

Appendix A: Slot Numbering Formulas

For a hand with `numPlayers` seated players and `holeCardCount` (2 for NLHE, 5 for PLO5):

```
holeSlot(playerIdx, card) = holeCardCount * playerIdx + card    (card: 0 .. holeCardCount-1)
flopSlot(k)               = holeCardCount * numPlayers + k      (k: 0, 1, 2)
turnSlot()                = holeCardCount * numPlayers + 3
riverSlot()               = holeCardCount * numPlayers + 4
```

All remaining positions up to 51 are unused burn cards.

Appendix B: Glossary

Mental poker — the general cryptographic problem of playing a fair card game over a network with no trusted third party to shuffle or deal.

SRA protocol — the Shamir-Rivest-Adleman (1979) construction solving mental poker for two players via commutative encryption; generalized here to N players.

Commutative encryption — an encryption scheme where the order of encrypting with multiple independent keys does not affect the ability to later decrypt by removing those keys in any order. Modular exponentiation to a shared prime modulus has this property.

Layer removal — the act of applying one’s own decryption exponent to a value in order to strip one (and only one) encryption layer, leaving any other players’ layers intact.

Showdown verification — the on-chain replay, using revealed keys and permutations, that confirms every value submitted during the deal was the genuine output of the claimed shuffle and encryption, catching any attempt to submit a dishonest deck array.

Side pot — a sub-pot created when players are all-in for different amounts, ensuring each player can only win from opponents they matched chips against.

Pot-limit — a betting structure (used here for PLO5) capping the maximum raise at the current size of the pot.